softstack

# Smart Contract Code Review And Security Analysis Report

| | |
|---|---|
| CUSTOMER | Trusset |
| SCOPE | Core Protocol |
| DATE | 11.05.2026 |

Made in Germany by softstack GmbH

# Table of Contents

# 1. Disclaimer

The audit makes no statements or warrantees about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

| Version / Date | Description |
|---|---|
| **0.1 (09.04.2026)** | Layout |
| **0.4 (13.04.2026)** | Automated Security Testing / Manual Security Testing |
| **0.5 (16.04.2026)** | Verify Claims |
| **0.9 (08.05.2026)** | Summary and Recommendation |
| **1.0 (11.05.2026)** | Re-check after mitigation and final document |

# 2. About the Project and Company

**Company:** Trusset UG

**Address:** Cuvrystr. 29, 10997 Berlin, Germany

**Website:** https://www.trusset.org

**GitHub:** https://github.com/trusset

**LinkedIn:** https://www.linkedin.com/company/trusset

Trusset is an open infrastructure protocol for financial institutions that turns static holdings such as stocks, commodities, real estate, and other real-world assets into programmable on-chain instruments. The platform provides modular APIs and self-contained smart contract suites that cover the full lifecycle of a tokenized asset, from issuance and corporate actions through 24/7 trading and credit, with MiCA- and eWpG-aligned compliance enforced at the contract level. Clients retain full ownership of their contracts, data, and governance, and integrate Trusset either as a white-label platform or directly via REST APIs and the TypeScript SDK.

At the core of Trusset is a credit-first architecture: every tokenized asset is designed to be immediately eligible as collateral for automated, overcollateralized credit lines. Risk parameters, liquidation thresholds, and investor eligibility are defined once and then enforced continuously by smart contracts, replacing manual underwriting and reconciliation. The protocol bundles a MiCA/eWpG-compliant KYC/AML identity registry with modular compliance rules, claim management, and machine-readable rejection codes, so the same identity layer can govern transfers, trading, and lending across all token types.

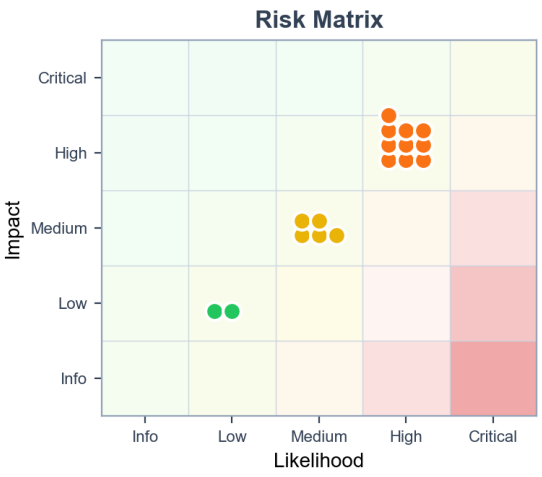
This audit covers six smart contract suites that together form the Trusset Core Protocol on EVM-compatible networks, built from four distinct codebases. The Stock Token License implements ERC-3643-style security tokens for regulated equity, including corporate actions, sub-issuer controls, and a force-transfer path for compliance interventions. The Commodity Token License provides ERC-20 tokens backed by physical reserves, with reserve-enforced minting, primary market sales, and configurable physical or cash redemption. The Stock Lending suite operates overcollateralized lending markets on top of the stock tokens, with an interest rate model, price oracle, insurance fund, Dutch-auction liquidations, and a shared liquidation router. The Commodity Orderbook suite provides hybrid custody for commodity token trading, with off-chain matching, on-chain settlement, batch trades, and liquidation routing under token-enforced compliance. Two additional suites share the same audited codebases: the Commodity Token Lending suite is identical to the Stock Lending codebase, and the Stock Orderbook License is identical to the Commodity Orderbook codebase. All findings and mitigations therefore apply equally to the paired suites.

All suites are written in Solidity, deployed as UUPS-upgradeable contracts, and built on OpenZeppelin's upgradeable libraries. Upgrades use a governance model that separates platform authority (the Trusset DAO) from issuer control, so changes require both parties to agree. Identity and compliance modules are duplicated and aligned across suites so that every token, market, and custody contract enforces the same KYC/AML rules at the boundary of every transfer.

# 3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

| Level | CVSS Score | Vulnerability | Required Action |
|---|---|---|---|
| CRITICAL | 9.0 – 10.0 | A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken. | Immediate action to reduce risk level. |
| HIGH | 7.0 – 8.9 | A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way. | Implementation of corrective actions as soon as possible. |
| MEDIUM | 4.0 – 6.9 | A vulnerability that could affect the desired outcome of executing the contract in a specific scenario. | Implementation of corrective actions in a certain period. |
| LOW | 0.1 – 3.9 | A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective. | Implementation of certain corrective actions or accepting the risk. |
| INFORMATIONAL | 0.0 | A vulnerability that has informational character but is not affecting any of the code. | An observation that does not determine a level of risk. |



# 4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

**Code review** that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

**Testing and automated analysis** that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

**Best practices review**, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

**Specific, itemized, actionable recommendations** to help you take steps to secure your smart contracts.

# 5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

## 5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

| Contract | Source | Commit |
|---|---|---|
| Stock Token | https://github.com/trusset/trusset-stock-token-license | d7a25b7f44752dcfbf8137059864526b2c89828a |
| Stock Lending | https://github.com/trusset/trusset-stock-lending | 3a342491fba93957de35e2ee6acae163207ed8fe |
| Commodity Token | https://github.com/trusset/trusset-commodity-token-license | da18d8b4b6e479301e38105e925513678477d75e |
| Commodity Orderbook | https://github.com/trusset/trusset-commodity-orderbook-license | c5134022ec732058a1d028c2d44875625f3efcd1 |
| Commodity Token Lending (same codebase as Stock Lending) | https://github.com/Paulilami/trusset-commodity-token-lending | 66acab027dd690fa1e6f42bc6d790e85462ef574 |
| Stock Orderbook (same codebase as Commodity Orderbook) | https://github.com/Paulilami/trusset-stock-orderbook-license | 89007d7dfa6056bb00403ed2b0af88d9da6cfa8b |

### trusset-commodity-orderbook-license-main

| File | Fingerprint (MD5) |
|---|---|
| ./trusset-commodity-orderbook-license-main/contracts/CommodityCustody.sol | 744c8d87ced6cb08339674b21948739d |

### trusset-commodity-token-license-main

| File | Fingerprint (MD5) |
| --- | --- |
| ./trusset-commodity-token-license-main/contracts/commodity-token/CommodityTokenFactoryUpgradeable.sol | 51650a488b4d68e21aa0a47ea9946fab |
| ./trusset-commodity-token-license-main/contracts/commodity-token/CommodityTokenUpgradeable.sol | cc4371ba85d81d6b83a067d6484a397e |
| ./trusset-commodity-token-license-main/contracts/compliance/BasicComplianceModule.sol | 96190d50a7499f650384f8958deda450 |
| ./trusset-commodity-token-license-main/contracts/compliance/IdentityRegistryUpgradeable.sol | 877fb79a0d08f43b823866754f8f68ff |
| ./trusset-commodity-token-license-main/contracts/interfaces/ICommodityToken.sol | 64d5f3a4a6656ba79cf8b2b76dbac3f2 |
| ./trusset-commodity-token-license-main/contracts/interfaces/IComplianceModule.sol | 790eaa7324041b61be1a9e6066753d85 |
| ./trusset-commodity-token-license-main/contracts/interfaces/IIdentityRegistry.sol | 2480d304cc94c499e189b71bc81a38b6 |
| ./trusset-commodity-token-license-main/contracts/interfaces/IPriceOracle.sol | be3ea5fbdcae675a5fe5d66a74277532 |
| ./trusset-commodity-token-license-main/contracts/primary-market/CommodityTokenSale.sol | f38cc357d77bcc26249b5fc1b65f60e1 |
| ./trusset-commodity-token-license-main/contracts/primary-market/CommodityTokenSaleFactory.sol | a5c30256d609b5f75f5e84b1c818fed2 |

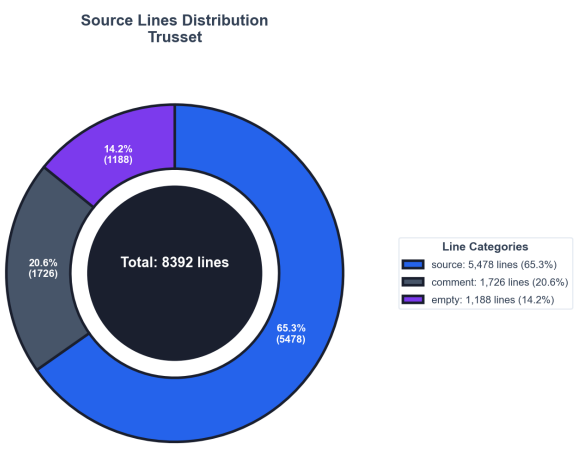
### trusset-stock-lending-main

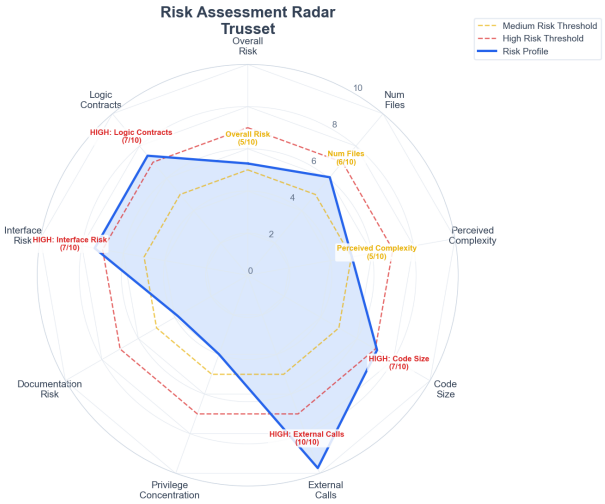
| File | Fingerprint (MD5) |
| --- | --- |
| ./trusset-stock-lending-main/contracts/LiquidationRouter.sol | 42f00c7d592270e916addc80f9506fe9 |
| ./trusset-stock-lending-main/contracts/StockInsuranceFund.sol | 4d17f5bfb936d8fb957a579461f95e36 |
| ./trusset-stock-lending-main/contracts/StockInterestRateModel.sol | 47706c226162d624e2b7c8cbb4f2a5e0 |
| ./trusset-stock-lending-main/contracts/StockLendingFactory.sol | 7f2063ebc38f4942f222a5dd72d820bb |
| ./trusset-stock-lending-main/contracts/StockLendingMarket.sol | e98d519c499ab801879adc62d764c742 |
| ./trusset-stock-lending-main/contracts/StockPriceOracle.sol | faac5ab515820ce4e1cb0d106810aad5 |

### trusset-stock-token-license-main

| File | Fingerprint (MD5) |
|---|---|
| ./trusset-stock-token-license-main/contracts/compliance/BasicComplianceModule.sol | 96190d50a7499f650384f8958deda450 |
| ./trusset-stock-token-license-main/contracts/compliance/IdentityRegistryUpgradeable.sol | 877fb79a0d08f43b823866754f8f68ff |
| ./trusset-stock-token-license-main/contracts/interfaces/IComplianceModule.sol | 790eaa7324041b61be1a9e6066753d85 |
| ./trusset-stock-token-license-main/contracts/interfaces/IIdentityRegistry.sol | dc7ad86edd514ff17a302c4e3dbaf3ca |
| ./trusset-stock-token-license-main/contracts/interfaces/ISecurityToken.sol | 2c3887bfbbccc43d9e8bcd06d990eaf1 |
| ./trusset-stock-token-license-main/contracts/stock-token/StockToken.sol | 858133461c62417ba4c5afb6c609dc04 |

## 5.2 Source Lines & Risk




## 5.3 Capabilities

| Module | Functions | Public | Restricted | Key Roles |
|---|---|---|---|---|
| BasicComplianceModule | 10 | 10 | 0 | - |
| CommodityCustody | 31 | 22 | 9 | admin |

| Module | Functions | Public | Restricted | Key Roles |
|---|---|---|---|---|
| CommodityTokenFactoryUpgradeable | 15 | 7 | 8 | admin |
| CommodityTokenSale | 16 | 16 | 0 | - |
| CommodityTokenSaleFactory | 7 | 4 | 3 | admin |
| CommodityTokenUpgradeable | 44 | 44 | 0 | - |
| ICommodityToken | 6 | 6 | 0 | - |
| IComplianceModule | 2 | 2 | 0 | - |
| IIdentityRegistry | 38 | 38 | 0 | - |
| IPriceOracle | 1 | 1 | 0 | - |
| ISecurityToken | 31 | 31 | 0 | - |
| IdentityRegistryUpgradeable | 56 | 16 | 40 | DEFAULT_ADMIN, KYC_PROVIDER, REGISTER_OPERATOR |
| LiquidationRouter | 17 | 7 | 10 | DEFAULT_ADMIN, OPERATOR |
| StockInsuranceFund | 9 | 5 | 4 | owner |
| StockInterestRateModel | 7 | 5 | 2 | owner |
| StockLendingFactory | 13 | 6 | 7 | owner |
| StockLendingMarket | 43 | 33 | 10 | DEFAULT_ADMIN, LIQUIDATOR, ISSUER |
| StockPriceOracle | 14 | 8 | 6 | owner |
| StockToken | 38 | 18 | 20 | ISSUER, SUB_ISSUER, CONTROLLER |

## 5.4 Dependencies / External Imports

| Import Path | Version |
|---|---|
| @openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol | 5.4.0 |

| Import Path | Version |
| --- | --- |
| @openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts/utils/structs/EnumerableSet.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/cryptography/EIP712Upgradeable.sol | 5.4.0 |
| @openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/ERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/IERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol | 5.4.0 |
| @openzeppelin/contracts/utils/cryptography/ECDSA.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/utils/cryptography/EIP712Upgradeable.sol | 5.4.0 |
| @openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/ERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/IERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol | 5.4.0 |
| @openzeppelin/contracts/utils/Pausable.sol | 5.4.0 |

| Import Path | Version |
|---|---|
| @openzeppelin/contracts/utils/ReentrancyGuard.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol | 5.4.0 |
| @openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/ERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/IERC20.sol | 5.4.0 |
| @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol | 5.4.0 |
| @openzeppelin/contracts/utils/ReentrancyGuard.sol | 5.4.0 |

## 5.5 Source Units in Scope

### trusset-commodity-orderbook-license-main

| File | Instructions | Functions | Lines | nSLOC | Comments |
|---|---|---|---|---|---|
| CommodityCustody.sol | 21 | 35 | 926 | 485 | 315 |

### trusset-commodity-token-license-main

| File | Instructions | Functions | Lines | nSLOC | Comments |
|---|---|---|---|---|---|
| CommodityTokenFactoryUpgradeable.sol | 10 | 16 | 353 | 200 | 105 |
| CommodityTokenUpgradeable.sol | 34 | 51 | 947 | 744 | 46 |
| CommodityTokenSaleFactory.sol | 6 | 7 | 154 | 83 | 50 |
| CommodityTokenSale.sol | 13 | 17 | 442 | 300 | 77 |
| BasicComplianceModule.sol | 4 | 5 | 170 | 83 | 61 |
| IdentityRegistryUpgradeable.sol | 16 | 32 | 736 | 461 | 185 |
| IPriceOracle.sol | 1 | 1 | 17 | 4 | 12 |
| IComplianceModule.sol | 0 | 1 | 39 | 17 | 20 |
| IIdentityRegistry.sol | 15 | 19 | 143 | 122 | 6 |
| ICommodityToken.sol | 5 | 6 | 31 | 15 | 14 |

### trusset-stock-lending-main

| File | Instructions | Functions | Lines | nSLOC | Comments |
|---|---|---|---|---|---|
| LiquidationRouter.sol | 11 | 18 | 337 | 259 | 31 |
| StockInsuranceFund.sol | 8 | 9 | 164 | 98 | 41 |
| StockPriceOracle.sol | 11 | 15 | 243 | 169 | 39 |
| StockInterestRateModel.sol | 3 | 8 | 125 | 85 | 25 |
| StockLendingMarket.sol | 33 | 60 | 1277 | 942 | 109 |
| StockLendingFactory.sol | 7 | 13 | 289 | 210 | 39 |

### trusset-stock-token-license-main

| File | Instructions | Functions | Lines | nSLOC | Comments |
|---|---|---|---|---|---|
| BasicComplianceModule.sol | 4 | 5 | 170 | 83 | 61 |
| IdentityRegistryUpgradeable.sol | 16 | 32 | 736 | 461 | 185 |
| StockToken.sol | 22 | 44 | 789 | 445 | 244 |
| ISecurityToken.sol | 29 | 31 | 111 | 73 | 24 |
| IComplianceModule.sol | 0 | 1 | 39 | 17 | 20 |
| IIdentityRegistry.sol | 15 | 19 | 154 | 122 | 17 |

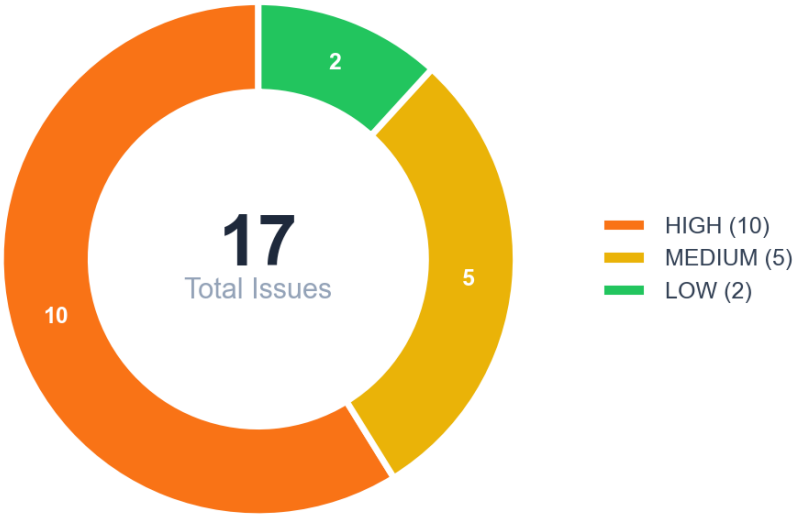
**Total (all contracts):** 284 instructions, 445 functions, 8392 lines, 5478 nSLOC

# 6. Scope of Work

This audit covers the Trusset Core Protocol smart contracts deployed on EVM-compatible networks. The scope spans six smart contract suites across four distinct codebases: the Stock Token License (ERC-3643 security tokens with compliance, identity registry, and corporate actions), the Stock Lending suite (overcollateralized lending markets, Dutch-auction liquidations, price oracle, insurance fund, and liquidation router), the Commodity Token License (ERC-20 commodity tokens with reserve enforcement and a primary market sale module), and the Commodity Orderbook License (hybrid orderbook custody with on-chain settlement and liquidation routing). Two additional suites share the same audited codebases: the Commodity Token Lending suite is identical to the Stock Lending codebase, and the Stock Orderbook License is identical to the Commodity Orderbook codebase. All findings and mitigations therefore apply equally to the paired suites. The audit focuses on access control, upgradeability, compliance enforcement, oracle integration, lending and liquidation correctness, settlement and custody safety, ERC-20 / token-transfer robustness, and general security of privileged operations.

## 6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



| No | Title | Severity | Status |
|---|---|---|---|
| 6.2.1 | Raw IERC20.transfer/transferFrom Without SafeERC20 | HIGH | FIXED |
| 6.2.2 | Empty Catch Swallows Lending Market Callback, Silent Fund Misrouting | HIGH | FIXED |
| 6.2.3 | Oracle Admin Can Crash Price via setMaxDeviation/updatePrice | HIGH | FIXED |
| 6.2.4 | removeLiquidity() Missing whenNotPaused, LP Bank Run During Emergency | HIGH | FIXED |
| 6.2.5 | Dutch Auction Stale Collateral Price Creates Systematic Bad Debt | HIGH | FIXED |
| 6.2.6 | Raw ecrecover Without Signature Malleability Check, Double Mint Execution | HIGH | FIXED |
| 6.2.7 | _calculateCashRedemptionAmount Scale-Factor Risk / Precision Mismatch | HIGH | FIXED |
| 6.2.8 | ISSUER_ROLE Can KYC-Bypass via addAuthorizedContract | HIGH | FIXED |
| 6.2.9 | CommodityCustody USDC Permanently Locked in Router | HIGH | FIXED |
| 6.2.10 | Stock Split Sub-Issuer Accounting Desynchronization | HIGH | FIXED |
| 6.2.11 | Dutch Auction Stale Price Window, Liquidation at Unfair Price | MEDIUM | FIXED |
| 6.2.12 | Settlement Operator Single-Key Unlimited Power | MEDIUM | FIXED |
| 6.2.13 | Oracle Timestamp Regression Allows Stale Price Replay | MEDIUM | FIXED |
| 6.2.14 | Stock Split Duplicate Holder Processing | MEDIUM | FIXED |
| 6.2.15 | emergencyWithdraw() No Timelock, Instant Fund Drainage | MEDIUM | FIXED |
| 6.2.16 | Raw IERC20.transfer() for USDC Payout in CommodityTokenUpgradeable | LOW | FIXED |
| 6.2.17 | CEI Violation in CommodityTokenSale, State Updates After External ETH Transfers | LOW | FIXED |

## 6.2 Manual and Automated Vulnerability Test

### 6.2.1 Raw IERC20.transfer/transferFrom Without SafeERC20

HIGH FIXED

### DESCRIPTION

The contract performs value transfers using raw IERC20.transfer / transferFrom calls and assumes a bool return or silent success. Several popular tokens (notably older USDT-style tokens) deviate from the ERC-20 return convention: they may not return a boolean, or they may revert on failure, or they can perform side-effects that break naive callers.

Root cause: missing use of a tolerant transfer wrapper; the code assumes a standard token implementation and does not handle no-return or unusual behaviors.

Impact: under real token permutations, transfers can silently fail or revert, leading to lost funds, stuck operations, or incorrect state being recorded (e.g., user balances updated while token transfer failed).

Evidence: PoC demonstrates a deposit flow that reverts or misbehaves using a USDT-like test token.

### CODE LOCATION

CommodityCustody.sol L384-418

```
function deposit(address token, uint256 amount) external nonReentrant
tokenSupported(token) {
if (amount == 0) revert ZeroAmount();
bool success = IERC20(token).transferFrom(msg.sender, address(this), amount);
if (!success) revert TransferFailed();
bytes32 key = _balanceKey(msg.sender, token);
_balances[key] += amount;
}
function withdraw(address token, uint256 amount) external nonReentrant {
...
bool success = IERC20(token).transfer(msg.sender, amount);
if (!success) revert TransferFailed();
}
```

### RECOMMENDATION

Replace all raw transfer / transferFrom usages with OpenZeppelin's SafeERC20 helpers:

Add unit/integration tests that use token mocks covering:

no-return ERC20 (does not return bool),

token that returns false,

token that reverts in transfer (ensure caller handles revert),

token that implements hooks that re-enter.

Add runtime sanity checks & events: after a transfer, emit an event recording transfer success and include revert handling to revert upstream if the token transfer fails.

### MITIGATION

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

### 6.2.2 Empty Catch Swallows Lending Market Callback, Silent Fund Misrouting

HIGH FIXED

#### DESCRIPTION

An empty try { ... } catch {} or a silent catch block is swallowing failures originating from external callbacks (e.g., lending market callbacks). When the external call fails, the contract hides the failure and proceeds, leaving funds or state in an inconsistent condition.

Root cause: the developer intended to be resilient to callback failures, but by swallowing all errors the contract fails to preserve invariants.

Impact: bad debt can remain uncovered, settlements can be incorrectly applied, and attackers can trigger silent failure paths to grief users or the protocol.

Evidence: PoC demonstrates an expected event is not emitted and a subsequent settlement path results in unrecoverable state.

#### CODE LOCATION

CommodityCustody.sol L689-703, L754-757 (settleLiquidation / settleLiquidationWithBuyer)

```
if (totalProceeds > 0) {
usdc.forceApprove(pending.lendingMarket, totalProceeds);
ILendingMarket(pending.lendingMarket).receiveLiquidationProceeds(liquidationId,
totalProceeds);
}
bool usdcSuccess = IERC20(usdc).transferFrom(msg.sender, liquidationRouter,
usdcReceived);
if (!usdcSuccess) revert TransferFailed();
// USDC already sent, no going back
if (liq.lendingMarket != address(0)) {
try ILendingMarket(liq.lendingMarket).receiveLiquidationProceeds(
liquidationId, usdcReceived
) {} catch {
// SILENT FAILURE, no event, no revert, nothing
}
}
```

#### RECOMMENDATION

Avoid silent catch. If an external callback can fail in acceptable cases, handle expected failure reasons explicitly. For unexpected failures, revert or emit a high-fidelity event containing the revert reason.

Example pattern:

Add defensive invariants and unit tests asserting that state remains consistent when callbacks revert. Add an operation-level circuit-breaker that refuses to proceed if required side-effects fail.

```
try externalContract.callback(...) {
// success path
} catch (bytes memory reason) {
revert(string(reason)); // or emit event and revert to preserve invariants
}
```

**MITIGATION**

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

### 6.2.3 Oracle Admin Can Crash Price via setMaxDeviation/updatePrice

HIGH FIXED

**DESCRIPTION**

Oracle functions setMaxDeviation and updatePrice can be used in quick succession to bypass intended circuit breakers and set price to artificially low/high levels.

**CODE LOCATION**

StockPriceOracle.sol L74-87, L184-188

```
function updatePrice(uint256 newPrice) external {
if (msg.sender != owner() && !authorizedSigners[msg.sender] && msg.sender !=
lendingMarket) {
revert UnauthorizedSigner();
}
if (newPrice < MIN_PRICE) revert InvalidPrice();
_checkDeviation(newPrice);
uint256 oldPrice = price;
price = newPrice;
updatedAt = block.timestamp;
}
function setMaxDeviation(uint256 newDeviationBps) external onlyOwner {
if (newDeviationBps == 0 || newDeviationBps > BPS_PRECISION) revert
InvalidDeviation();
maxDeviationBps = newDeviationBps;
}
```

**RECOMMENDATION**

Timelock sensitive oracle configuration changes.

Add two-step proposals with delay for setMaxDeviation and similar settings.

Add upper/lower bound sanity checks and multi-oracle verification before applying large deviations.

**MITIGATION**

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.4 removeLiquidity() Missing whenNotPaused, LP Bank Run During Emergency

HIGH FIXED

**DESCRIPTION**

removeLiquidity() is callable during paused/emergency conditions, allowing LPs to withdraw in a way that can destabilize emergency handling.

**CODE LOCATION**

StockLendingMarket.sol L335

```
// StockLendingMarket.sol L335
function addLiquidity(uint256 amount) external nonReentrant whenNotPaused {
// StockLendingMarket.sol L363, NO whenNotPaused!
function removeLiquidity(uint256 shares) external nonReentrant {
if (shares == 0) revert InvalidAmount();
LiquidityProvider storage lp = liquidityProviders[msg.sender];
if (lp.shares < shares) revert InsufficientShares();
...
usdc.safeTransfer(msg.sender, usdcAmount);
function removeLiquidity(uint256 shares) external nonReentrant whenNotPaused {
```

**RECOMMENDATION**

Add whenNotPaused modifier and nonReentrant guard. Add tests verifying paused state blocks removeLiquidity().

**MITIGATION**

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.5 Dutch Auction Stale Collateral Price Creates Systematic Bad Debt

HIGH FIXED

### DESCRIPTION

The auction workflow relies on a collateral price snapshot that can remain unchanged throughout the auction lifecycle instead of forcing a fresh pricing check when liquidation is finalized. If market conditions move materially while the auction is open, the protocol can settle against an outdated collateral valuation.

Root cause: the liquidation path treats the auction's stored pricing inputs as authoritative even after time has passed, without a freshness bound, revalidation step, or invalidation path when oracle conditions change.

Impact: liquidators can acquire collateral at systematically stale prices, leaving the protocol with undercollateralized debt that is not fully covered by auction proceeds. In stressed markets this compounds into recurring lender losses, insurance fund depletion, and protocol-wide bad debt.

### CODE LOCATION

StockLendingMarket.sol ~L1207-1220; getAuctionPrice L702-712

```
// StockLendingMarket.sol ~L1207-1220
uint256 startPrice = (currentPrice * config.auctionStartPremium) / PRECISION;
uint256 endPrice = (currentPrice * config.auctionMinPremium) / PRECISION;
dutchAuctions[auctionId] = DutchAuction({
startPrice: startPrice,
endPrice: endPrice,
startTime: block.timestamp,
endTime: block.timestamp + config.auctionDuration,
...
});
// getAuctionPrice L702-712
function getAuctionPrice(uint256 auctionId) public view returns (uint256) {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) return 0;
if (block.timestamp >= auction.endTime) return auction.endPrice;
uint256 elapsed = block.timestamp - auction.startTime;
uint256 duration = auction.endTime - auction.startTime;
uint256 priceDrop = auction.startPrice - auction.endPrice;
uint256 currentDrop = (priceDrop * elapsed) / duration;
return auction.startPrice - currentDrop;
}
```

### RECOMMENDATION

Recompute the final settlement price at execution time using the latest validated oracle value or a guarded TWAP, and revert if the live price differs from the stored auction price beyond a configured tolerance.

Add an auction freshness guard such as maxAuctionAge, and expose a resetAuction or restartAuction path that invalidates stale auctions instead of allowing them to settle on expired assumptions.

Introduce liquidation safety rails: minimum recovery ratio checks, circuit breakers for large oracle moves, and explicit bad-debt accounting when proceeds cannot cover obligations.

Add tests that simulate oracle drift during the auction window, large downward repricing between auction creation and settlement, and repeated stale settlements to confirm the protocol either reprices or aborts safely.

**MITIGATION**

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.6 Raw ecrecover Without Signature Malleability Check, Double Mint Execution

HIGH FIXED

**DESCRIPTION**

The contract performs signature verification using raw ecrecover and does not canonicalize or validate the s value and v parameters, which enables signature malleability and replay / double-execution of signed operations.

Root cause: using raw ecrecover without using a vetted helper (ECDSA from OpenZeppelin) and without checking s is in the lower half of the curve and v is valid.

Impact: an attacker can craft or modify signatures to satisfy verification multiple times (double mint, double redeem), causing unauthorized token issuance or fund transfers.

Evidence: PoC executed locally after enabling allowUnlimitedContractSize in the project's hardhat.config.ts; observed double mint (supply doubled after malleable signature execution).

**CODE LOCATION**

CommodityTokenUpgradeable.sol L497-513 (executeMintRequestConditional)

```
// L497-513, executeMintRequestConditional
function executeMintRequestConditional(
uint256 requestId, uint256 deadline, uint8 v, bytes32 r, bytes32 s
) external whenNotPaused nonReentrant {
MintRequest storage request = mintRequests[requestId];
// ... checks ...
address signer = ecrecover(_hashTypedDataV4(structHash), v, r, s); // ← RAW
ecrecover
if (signer == address(0) || (signer != mainIssuer && !delegates[signer])) revert
InvalidSignature();
request.status = RequestStatus.APPROVED; // ← OVERWRITES EXECUTED → APPROVED !!!
_executeMintRequest(requestId); // ← guard at L518 sees APPROVED, not EXECUTED
}
```

```
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
address signer = ECDSA.recover(_hashTypedDataV4(structHash), v, r, s);
```

#### RECOMMENDATION

Replace raw ecrecover usage with OpenZeppelin ECDSA utilities:

If ecrecover is used directly, enforce canonical s and v checks:

Add regression tests that replay signatures with s-flipped values and ensure only one canonical signature validates. Add monitoring to detect duplicate-signature executions.

```
bytes32 txHash = keccak256(...);
address signer = ECDSA.recover(ECDSA.toEthSignedMessageHash(txHash), signature);
require(uint256(s) <= SECP256K1N_DIV_2, "InvalidSignatureS");
require(v == 27 || v == 28, "InvalidSignatureV");
```

#### MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

### 6.2.7 _calculateCashRedemptionAmount Scale-Factor Risk / Precision Mismatch

HIGH FIXED

#### DESCRIPTION

Fixed-point arithmetic mismatches and coefficient scaling in _calculateCashRedemptionAmount can cause rounding errors, off-by-magnitude miscalculations, and overflows when handling extreme values.

Root cause: Inconsistent scaling factors, missing overflow guards, and insufficient unit tests that exercise extreme ranges.

Impact: Under/overpayment during redemptions; possible arithmetic exploits when combined with other logic.

#### CODE LOCATION

CommodityTokenUpgradeable.sol L730-736

```
// CommodityTokenUpgradeable.sol L730-736
function _calculateCashRedemptionAmount(uint256 tokenAmount) private view returns
(uint256) {
(uint256 price, uint8 priceDecimals, uint256 timestamp) =
IPriceOracle(cashRedemptionOracle).getPrice();
if (block.timestamp - timestamp > maxOracleAge) revert OraclePriceStale();
```

```
uint256 scaleFactor = 10 ** (uint256(18) + uint256(priceDecimals) -
uint256(usdcDecimals));
return (tokenAmount * price) / scaleFactor;
}
require(price > 0, "Oracle zero price");
require(uint256(18) + uint256(priceDecimals) >= uint256(usdcDecimals), "Scale
underflow");
// Use mulDiv for overflow protection
```

#### RECOMMENDATION

Adopt a single fixed-point library pattern (e.g., FixedPointMath), define BASE = 1e18, and normalize all internal arithmetic to the BASE.

Add explicit require checks for denominator non-zero and range checks for inputs.

Add fuzz tests covering min/max values and cross-check expected vs. high-precision off-chain results.

#### MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

### 6.2.8 ISSUER_ROLE Can KYC-Bypass via addAuthorizedContract

HIGH FIXED

#### DESCRIPTION

The ISSUER_ROLE can call addAuthorizedContract, potentially authorizing contracts that bypass KYC checks.

Root cause: Role-scoped permissions that grant overly broad authority to add trusted contracts without cross-checks.

Impact: An attacker with ISSUER_ROLE can authorize malicious contracts and bypass compliance checks.

#### CODE LOCATION

StockToken.sol L133-135

```
// StockToken.sol L133-135
function addAuthorizedContract(address contractAddr) external
onlyRole(ISSUER_ROLE) {
if (contractAddr == address(0)) revert InvalidAddress();
_authorizedContracts[contractAddr] = true;
}
// StockToken.sol L469-476 (forceTransfer)
```

```
function forceTransfer(address from, address to, uint256 amount, bytes32 reason)
external override onlyRole(CONTROLLER_ROLE) nonReentrant
{
...
// Recipient must be verified even for forced transfers
if (!_authorizedContracts[to]) {
(bool verified,) = _identityRegistry.isVerified(to);
if (!verified) revert IdentityNotVerified();
}
// ↑ If 'to' is in _authorizedContracts → identity check SKIPPED
if (!_authorizedContracts[from]) { /* identity check */ }
if (!_authorizedContracts[to]) { /* identity check */ }
```

#### RECOMMENDATION

Restrict addAuthorizedContract to a higher-privileged role or add secondary validation, e.g., onlyRole(DEFAULT_ADMIN_ROLE) plus off-chain review + timelock.

Verify the target is a contract (not EOA) and that it implements expected interface selectors.

Tests:

Ensure ISSUER_ROLE cannot add unauthorized contract addresses.

#### MITIGATION

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

### 6.2.9 CommodityCustody USDC Permanently Locked in Router

HIGH FIXED

#### DESCRIPTION

Funds sent to the router can be trapped when downstream callbacks fail and the router lacks compensating refund/path recovery.

Root cause: Router performs transfer/call to downstream contracts without robust fallback or accounting of owed balances.

Impact: User funds permanently locked or lost.

#### CODE LOCATION

CommodityCustody.sol L689-703; StockLendingMarket.sol

```
// CommodityCustody.sol L689-703
function settleLiquidation(uint256 liquidationId, uint256 usdcReceived, address
tokenRecipient)
external onlyOperator nonReentrant
```

```
{
...
liq.settled = true;
// Transfer tokens to buyer
bool tokenSuccess = IERC20(token).transfer(tokenRecipient, tokenAmount);
if (!tokenSuccess) revert TransferFailed();
// USDC sent to liquidationRouter, NOT kept in CommodityCustody
bool usdcSuccess = IERC20(usdc).transferFrom(msg.sender, liquidationRouter,
usdcReceived);
if (!usdcSuccess) revert TransferFailed();
// Callback called FROM CommodityCustody (msg.sender = CommodityCustody)
if (liq.lendingMarket != address(0)) {
try ILendingMarket(liq.lendingMarket).receiveLiquidationProceeds(
liquidationId, usdcReceived
) {} catch {
// ← SILENT FAILURE: proceeds mismatch swallowed
}
}
}
// StockLendingMarket.sol
function receiveLiquidationProceeds(uint256 liquidationId, uint256 amount)
external nonReentrant {
if (msg.sender != liquidationContract) revert OnlyLiquidationContract();
usdc.safeTransferFrom(msg.sender, address(this), amount); // pulls from
CommodityCustody
```

**RECOMMENDATION**

Track owed balances explicitly and use pull-payment patterns for risky transfers.

On callback failure, record owed balance and expose claim() for users to recover funds.

Tests:

Simulate downstream callback failures and assert router records owed amounts and allows recovery.

**MITIGATION**

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

### 6.2.10 Stock Split Sub-Issuer Accounting Desynchronization

HIGH FIXED

### DESCRIPTION

stockSplit() doubles holder balances but never adjusts _subIssuerMinted or _subIssuerCap, so the token's sub-issuer accounting becomes permanently stale immediately after any forward split.

Root cause: The split loop mints extra balance directly to holders and updates the split multipliers, but it never synchronizes the sub-issuer bookkeeping that redeemFrom() and _issue() rely on for redemption and cap enforcement.

Impact: After a forward split, sub-issuers can be blocked from redeeming tokens that now exist in circulation, and they can also be blocked from issuing within their true post-split cap because the contract still enforces the stale pre-split counters.

### CODE LOCATION

StockToken.sol L394-405, L508-546, L670-686

```
function redeemFrom(address account, uint256 amount, bytes32 reason)
external override onlyRole(SUB_ISSUER_ROLE) whenNotPaused nonReentrant
{
uint256 outstanding = _subIssuerMinted[msg.sender];
if (amount > outstanding) revert SubIssuerRedemptionExceeded();
_subIssuerMinted[msg.sender] = outstanding - amount;
_redeem(account, amount);
}
function stockSplit(uint256 numerator, uint256 denominator, address[] calldata
holders)
external override onlyRole(ISSUER_ROLE) whenNotPaused
{
...
for (uint256 i = 0; i < holders.length;) {
...
if (diff > 0) {
_mint(holder, diff);
}
unchecked { ++i; }
}
// no `_subIssuerMinted` or `_subIssuerCap` update anywhere in the split path
}
function _issue(address subIssuer, address to, uint256 amount, bytes32 reason)
internal {
uint256 cap = _subIssuerCap[subIssuer];
uint256 newMinted = _subIssuerMinted[subIssuer] + amount;
if (cap > 0 && newMinted > cap) revert SubIssuerCapExceeded();
_subIssuerMinted[subIssuer] = newMinted;
}
```

### RECOMMENDATION

Batch updates within the same transaction where feasible; use explicit cross-checks after updates to ensure masterSupply == sum(subIssuerSupplies).

If cross-contract atomicity is not possible, implement two-phase commit: prepare → finalize, with reconciliation and an automated repair tool that raises alerts on drift.

Add monitoring and on-chain invariant asserts that automatically pause relevant functions if desync exceeds a small threshold.

Tests:

Simulate concurrent split/issue flows and ensure invariants hold or trigger safe-fail behavior.

#### MITIGATION

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

### 6.2.11 Dutch Auction Stale Price Window, Liquidation at Unfair Price

MEDIUM FIXED

#### DESCRIPTION

The auction uses a stored endPrice computed earlier and does not re-check oracle price at settlement time, enabling bidders to win auctions priced on stale data.

Impact: liquidations at stale (unfavorable) prices cause loss to lenders/borrowers and profit to attackers; PoC shows a 40% discount exploitation scenario.

#### CODE LOCATION

StockLendingMarket.sol L612-637, L702-712

```
function bidOnAuction(uint256 auctionId, uint256 collateralAmount) external
nonReentrant whenNotPaused {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) revert AuctionNotActive();
if (collateralAmount > auction.collateralRemaining) {
collateralAmount = auction.collateralRemaining;
}
uint256 currentPrice = getAuctionPrice(auctionId);
uint256 usdcRequired = (collateralAmount * currentPrice) / stockTokenScale;
usdc.safeTransferFrom(msg.sender, address(this), usdcRequired);
IERC20(address(stockToken)).safeTransfer(msg.sender, collateralAmount);
}
function getAuctionPrice(uint256 auctionId) public view returns (uint256) {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) return 0;
if (block.timestamp >= auction.endTime) return auction.endPrice;
...
}
```

#### RECOMMENDATION

Recompute final settlement price at execution time using the latest oracle/TWAP.

Add a price-freshness check comparing endPrice with current oracle price and revert if divergence exceeds a safe threshold.

Add integration tests that simulate oracle divergence and assert settlement reverts or uses corrected price.

#### MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.12 Settlement Operator Single-Key Unlimited Power

MEDIUM FIXED

#### DESCRIPTION

The settlement operator currently has broad, unilateral permissions that allow it to finalize settlements, route funds, and change bookkeeping without multi-party checks. These operations affect user balances and fund custody, but are executed by a single EOA or key.

Root cause: Role design conflates low-impact bookkeeping tasks with high-impact fund movement. The code lacks separation of duties and fails to require multi-sig, timelock, or gating conditions for high-risk flows.

Impact: If the operator key is compromised, an attacker can mis-route funds, withdraw assets, or perform settlements that unjustly credit/debit user positions, causing asset loss, reputational damage, and recoverability challenges.

#### CODE LOCATION

CommodityCustody.sol L40-47, L331-346, L434-513

```
address public settlementOperator;
modifier onlyOperator() {
if (msg.sender != settlementOperator && msg.sender != admin) revert
Unauthorized();
_;
}
function freezeUser(address user) external onlyOperator {
if (user == address(0)) revert ZeroAddress();
_frozenUsers[user] = true;
}
function lockBalance(address user, address token, uint256 amount, bytes32 lockId)
external onlyOperator tokenSupported(token)
{ ... }
```

```
function settleTrade(..., uint256 baseAmount, uint256 quoteAmount, bool
makerIsBuyer)
external onlyOperator nonReentrant
{
...
_executeTransfer(maker, taker, baseToken, baseAmount);
_executeTransfer(taker, maker, quoteToken, quoteAmount);
}
```

**RECOMMENDATION**

Re-scope roles: split OPERATOR into SETTLEMENT_EXECUTOR (non-custodial, idempotent bookkeeping) and FUNDS_CONTROLLER (custodial, restricted). Limit FUNDS_CONTROLLER actions by timelock and multisig.

Require multi-sig (Gnosis Safe or equivalent) for transfers or settlement operations above a configurable threshold. Implement thresholds in code and enforce via require checks referencing a treasury policy contract.

Add precondition checks: verify invariant sums (pre- and post-settlement totalSupply/totalAssets equality) and assert them; if violated, revert and emit an InvariantViolation event with diagnostic context.

Introduce an emergency freeze role that can pause settlement execution but cannot perform fund movements itself.

Tests:

Simulate compromised operator: assert that large withdrawals require multi-sig and fail when called by single key.

Unit tests for invariants: run settlements in randomized sequences and assert totals remain balanced after each settlement.

Integration test: attempt to call settlement path with operator EOA and ensure multisig gating prevents execution for >threshold flows.

**MITIGATION**

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

The settlementOperator is now pointed at a multisig wallet combined with a timelock controller instead of a single EOA. The mitigation was applied at the deployment level rather than through a contract-code change.

### 6.2.13 Oracle Timestamp Regression Allows Stale Price Replay

MEDIUM FIXED

#### DESCRIPTION

Oracle updates accept an updatedAt field from off-chain sources; if the provider allows non-monotonic timestamps, an attacker can replay stale signed price updates or submit earlier timestamps that pass naive checks.

Root cause: Relying on externally-supplied timestamps without strict monotonicity checks or embedded monotonic counters.

Impact: Attackers can replay previously-signed price messages to force stale prices to be accepted, enabling profit via arbitrage or causing inaccurate settlements.

#### CODE LOCATION

StockPriceOracle.sol L96-120 (updatePriceWithSignature)

#### RECOMMENDATION

Require signed messages to include a monotonic nonce/sequence number or a block-based counter, and validate nonce > lastNonce.

Accept signed price messages only if updatedAt >= lastAcceptedTimestamp and reject regressions.

Where possible, cross-check with block.timestamp derived freshness or require a small acceptable delta.

Tests:

Send signed updates with regressing updatedAt and ensure contracts reject them.

Integration: verify cross-oracle monotonic enforcement in multi-oracle setups.

#### MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.14 Stock Split Duplicate Holder Processing

MEDIUM FIXED

#### DESCRIPTION

Processing holder arrays with duplicate addresses causes double-counting and incorrect balance allocations during split operations.

Root cause: Using raw arrays of addresses without canonicalization (dedupe) and performing per-address operations that assume uniqueness.

Impact: Inflated balances and supply inconsistencies; potential imbalance resulting in mint/burn mismatch.

#### CODE LOCATION

StockToken.sol L508-546

```
function stockSplit(uint256 numerator, uint256 denominator, address[] calldata
holders)
external override onlyRole(ISSUER_ROLE) whenNotPaused
{
...
for (uint256 i = 0; i < holders.length;) {
address holder = holders[i];
uint256 currentBalance = balanceOf(holder);
if (currentBalance > 0) {
uint256 newBalance = (currentBalance * numerator) / denominator;
uint256 diff = newBalance - currentBalance;
if (diff > 0) {
_mint(holder, diff);
}
}
unchecked { ++i; }
}
}
```

**RECOMMENDATION**

Normalize holder sets: sort and deduplicate holder addresses before processing splits. Use a memory hashmap or mark-and-sweep approach to handle large lists gas-efficiently.

Add checks ensuring sum(post-split balances) == sum(pre-split balances) and revert on mismatch.

Tests:

Supply edge-case test with duplicate addresses and assert invariants.

Gas regression: ensure dedup approach scales for large holder lists.

**MITIGATION**

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

### 6.2.15 emergencyWithdraw() No Timelock, Instant Fund Drainage

MEDIUM FIXED

**DESCRIPTION**

An emergencyWithdraw() function permits immediate withdrawals by an admin without timelock, allowing instant large fund movements.

Root cause: Privileged function lacking governance/time-delays and threshold checks.

Impact: Compromise or misuse of admin key results in direct draining of funds.

### CODE LOCATION

StockInsuranceFund.sol L91-99

```
// StockInsuranceFund.sol:91-99
function emergencyWithdraw(uint256 amount, address to) external onlyOwner
nonReentrant {
if (to == address(0)) revert InvalidAddress(); // has zero-addr check
if (amount == 0) revert ZeroAmount(); // has zero-amount check
uint256 balance = usdc.balanceOf(address(this));
if (amount > balance) revert InsufficientBalance(); // has balance check
usdc.safeTransfer(to, amount); // NO timelock delay
totalWithdrawn += amount; // NO withdrawal cap
emit EmergencyWithdraw(amount, to); // NO prior announcement
}
// Missing: timelock delay, withdrawal cap, multi-sig requirement, cooldown period
// Missing: minimum reserve ratio to protect pending bad debt coverage
// StockLendingFactory.sol:180,204
// Insurance fund initialized with owner = factory
abi.encodeCall(StockInsuranceFund.initialize, (usdc, address(this)))
// Then transferred to admin (arbitrary EOA)
StockInsuranceFund(insuranceFund).transferOwnership(admin);
// Require a 2-step process: announce → wait → execute
uint256 public constant EMERGENCY_WITHDRAW_DELAY = 48 hours;
mapping(bytes32 => uint256) public pendingWithdrawals;
function announceEmergencyWithdraw(uint256 amount, address to) external onlyOwner
{
bytes32 id = keccak256(abi.encode(amount, to));
pendingWithdrawals[id] = block.timestamp + EMERGENCY_WITHDRAW_DELAY;
emit EmergencyWithdrawAnnounced(amount, to, block.timestamp +
EMERGENCY_WITHDRAW_DELAY);
}
function executeEmergencyWithdraw(uint256 amount, address to) external onlyOwner
nonReentrant {
bytes32 id = keccak256(abi.encode(amount, to));
require(pendingWithdrawals[id] != 0 && block.timestamp >= pendingWithdrawals[id],
"Not ready");
delete pendingWithdrawals[id];
// ... existing withdrawal logic
}
```

### RECOMMENDATION

Gate emergency withdrawals behind TimelockController for large transfers and require multi-sig approval for transfers above configurable thresholds.

Implement per-period caps (e.g., max withdraw per 24h) and a delay window where large withdrawals are subject to community review.

Add extensive event logging and an on-chain audit trail for emergency flows.

Tests:

Attempt emergency withdrawals as single admin and ensure timelock/multisig gating prevents immediate execution.

#### MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

### 6.2.16 Raw IERC20.transfer() for USDC Payout in CommodityTokenUpgradeable

LOW FIXED

#### DESCRIPTION

Similar to F-003: raw transfers used for USDC payouts assume standard ERC-20 boolean return and fail on nonconforming tokens.

Impact: payout failures and stuck funds.

#### CODE LOCATION

CommodityTokenUpgradeable.sol L718

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
using SafeERC20 for IERC20Metadata;
// Line 718: Replace
// bool success = IERC20Metadata(usdcToken).transfer(request.account, usdcAmount);
// require(success, "USDC_TRANSFER_FAILED");
// With:
IERC20Metadata(usdcToken).safeTransfer(request.account, usdcAmount);
```

#### RECOMMENDATION

Use SafeERC20 wrappers and add tests mocking nonconforming USDC-like implementations.

#### MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

### 6.2.17 CEI Violation in CommodityTokenSale, State Updates After External ETH Transfers

LOW FIXED

### DESCRIPTION

The contract performs an external ETH transfer (or token transfer) to a recipient before updating internal accounting that records purchase/sale state. This order violates Checks-Effects-Interactions and opens the contract to reentrancy or inconsistent state if the external call reenters.

Root cause: Business flow updates external balances prior to committing internal state changes (effects), relying on the external target to behave benignly.

Impact: Malicious recipients can reenter and call into contract functions expecting a post-transfer state, causing duplicates, refunds, or drained state transitions.

### CODE LOCATION

CommodityTokenSale.sol L245-332 (purchase)

### RECOMMENDATION

Reorder operations to follow CEI: perform all require checks, update internal state (balances, totalSupply), then interact with external contracts or transfer ETH/tokens.

Add nonReentrant modifier (OpenZeppelin ReentrancyGuard) to all externally-facing state-mutating functions.

Run fuzz and reentrancy tests that attempt reentry during external transfer.

Add a defensive fallback: when an external transfer fails, revert with a clear revert reason and avoid partial state application.

### MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

# 7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the Trusset team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 17 issues, classified as follows:

- No critical issues were found
- 10 high severity issues were found
- 5 medium severity issues were found
- 2 low severity issues were found
- No informational issues were found

The Trusset team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

# 8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over $100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: https://softstack.io

# 9. Glossary

| Term | Definition |
|---|---|
| **AML** | Anti-Money Laundering |
| **CEI** | Checks-Effects-Interactions pattern for reentrancy prevention |
| **CVSS** | Common Vulnerability Scoring System |
| **Custody** | Wallet-agnostic contract holding user balances and enforcing settlement rules |
| **DAO** | Decentralized Autonomous Organization |
| **DeFi** | Decentralized Finance |
| **Dutch Auction** | Descending-price auction used to liquidate collateral |
| **ERC-20** | Ethereum token standard for fungible tokens |
| **ERC-3643** | Permissioned security-token standard with on-chain identity and transfer compliance |
| **EVM** | Ethereum Virtual Machine |
| **Identity Registry** | On-chain registry storing verified investor identities and claims |
| **KYC** | Know Your Customer; identity verification process |
| **LP** | Liquidity Provider in a lending or trading market |
| **Liquidation Router** | Shared contract that routes liquidation proceeds between lending markets and custody |
| **MiCA** | Markets in Crypto-Assets Regulation (EU) |
| **OZ** | OpenZeppelin, widely-used Solidity library for secure smart contracts |
| **Oracle** | On-chain price feed sourced from external data providers |
| **SafeERC20** | OpenZeppelin helper for safely interacting with non-standard ERC-20 tokens |
| **TWAP** | Time-Weighted Average Price |
| **TrussetDAO** | Trusset governance entity that co-signs platform-level contract upgrades |
| **USDC** | USD Coin; fiat-backed stablecoin used for settlements and lending |
| **UUPS** | Universal Upgradeable Proxy Standard |
| **eWpG** | Elektronische Wertpapiergesetz; German electronic securities law |